Preston-Check — User Manual

Pre-deployment security audit for fintech, end-to-end

2026-05-04

User Manual

Preston-Check runs 294 hand-curated security checks against your source code. It's an open-source command-line tool that runs locally and never sends source code anywhere unless you explicitly opt in. This manual covers every flag, every output format, every integration path, and the conventions that the tool depends on. If you've used it casually already, the section that pays back the most is "Filters" — most production users run only a slice of the catalog tuned to their compliance scope.

Install

Five channels, all kept in lockstep on every release. Pick the one that matches your environment.

Homebrew (macOS, or Linux with brew):

```
brew tap preston-check/preston-check
brew install preston-check
```

Docker (any system with Docker):

```
docker run --rm -v "$(pwd):/src" prestoncheck/scan:latest
```

Mount your project at `/src`. Add `-v ~/.preston-check:/home/preston/.preston-check` to share license files into the container.

Curl-bash installer (works anywhere with curl + tar + sha256):

```
curl -fsSL https://get.preston-check.com/install.sh | sh
```

Verifies the SHA-256 of the release tarball before extracting, installs the catalog under `$PREFIX/share/preston-check`, drops a shim at `$PREFIX/bin/preston-check`. Defaults to `/usr/local`; override with `PRESTON_PREFIX=$HOME/.local`. Pin a version with `PRESTON_VERSION=v1.7.5`.

GitHub Action (in your CI):

```
- uses: preston-check/scan-action@v1
  with:
    fail-on: high
    report-path: preston-check-report.md
```

Source clone (everything is Apache 2.0):

```
git clone https://github.com/preston-check/preston-check
cd preston-check
./preston-check.sh --help
```

Your first scan

In any directory:

```
preston-check
```

You'll see a banner reporting the detected language, license tier, OSS-license exemption status, telemetry status, and run mode. Each check prints a colored result line — green PASS, red FAIL, yellow WARN, blue SKIP — and the summary at the end shows totals plus a single A–F letter grade and a 0–100 score.

Save the report to disk:

```
preston-check --report security-audit.md
```

The Markdown is suitable for code-review attachments and PR comments. If `pandoc` plus a PDF renderer (Chrome or `wkhtmltopdf`) are available, a PDF version lands next to the Markdown.

Run modes

Light — P-01 through P-20, the original 20 categories of core fintech checks. Completes in under 30 seconds on a typical medium-sized repo. Ideal for pre-commit hooks and watch-mode development:

```
preston-check --light
```

Full — every check in the catalog (default mode). Three to four minutes on a large repo. Run this on every push in CI and before every release:

```
preston-check --full
```

Critical-only — only the highest-severity checks across the entire catalog (~12 second runtime). The fast-core run for git pre-commit hooks where every second matters:

```
preston-check --critical-only
```

High and up — critical + high severity (~73 checks). The recommended CI-blocking severity tier:

```
preston-check --high-and-up --ci
```

Filters

Filter by framework, category, or severity. All combinations work together (intersection).

By framework — produce a per-framework audit report. The framework filter matches against each check's metadata, so any framework name that appears in the catalog works:

```
preston-check --framework "PCI-DSS"      --report pci-audit.md
preston-check --framework MiCA            --report mica-audit.md
preston-check --framework DORA            --report dora-audit.md
preston-check --framework "NYDFS"         --report nydfs-audit.md
preston-check --framework "OWASP-LLM"     --report llm-audit.md
preston-check --framework "OWASP-SC-Top-10:2025" --report sc-audit.md
```

The full list of 33 frameworks is in `docs/all-frameworks.md`.

By category — pick the kind of check rather than the framework:

```
preston-check --code-only       # pure source-code analysis
preston-check --docs-only       # policy / evidence documentation
preston-check --infra-only      # infrastructure config only
preston-check --live-only       # SSH-based production log checks
```

By severity — short-circuit when you want to focus on highest-impact issues:

```
preston-check --severity critical,high
preston-check --critical-only      # alias for --severity critical
preston-check --high-and-up        # alias for --severity critical,high
```

CI integration

Every CI surface gets the same exit codes and the same report. The `--ci` flag turns FAILs into a non-zero exit (default 1), so your CI job fails when blocking issues land. `--ci-soft` always exits 0 — use this when you want CI to inspect the report itself rather than relying on exit codes.

GitHub Actions, GitLab CI, CircleCI, and a Git pre-commit hook example each ship in `examples/`. Drop into your project as-is.

AI augmentation

When `ANTHROPIC_API_KEY` or `OPENAI_API_KEY` is set in your environment, two flags unlock AI-augmented findings:

```
preston-check --ai-augment      # adds AI explanations + false-positive flags
preston-check --ai-fix          # also generates suggested patches per finding
```

For each FAIL/WARN finding (capped at 5 per check to bound scan time), the AI subsystem sends the file:line:content plus 10–15 surrounding lines to your configured LLM and includes the response inline in the report addendum. Per-finding responses are cached under `~/.preston-check/ai-cache/` so reruns over unchanged code are free. Local Ollama is also supported via `PRESTON_AI_PROVIDER=ollama`.

Both flags are unconditional no-ops under `--airgap`. You bring your own API key — there's no usage routing through Preston-Check's servers.

Telemetry

Off by default. Three equivalent ways to opt in:

```
preston-check --telemetry-opt-in            # per-run flag
export PRESTON_TELEMETRY=1                  # env var
echo 'telemetry: opt_in' >> .preston-check.yml   # config file
```

The full payload is documented at `docs/telemetry.md`. Ten fields, all aggregate: tool version, license tier, primary language, repo URL hash, the four pass/fail/warn/skip counts, a calculated score, and a UTC timestamp. No source code, file paths, file names, or specific check names are ever sent. The opt-in payload feeds the annual State of Fintech Security report and the peer-comparison percentiles in the SaaS dashboard.

`--airgap` forbids all network calls and overrides every opt-in.

Configuration file

For repeatable runs, create `.preston-check.yml` at the repo root:

```
app_name: my-payment-service
source_dir: .
log_dir: /var/log/myapp
ssh_host: prod-server-01
api_base_url: https://api.example.com
redis_host: localhost
db_host: db.example.com
```

Then run:

```
preston-check --config .preston-check.yml
```

Only `source_dir` is required. Live-monitoring checks (P-10) need `ssh_host`. Other fields are used by specific check categories and degrade to SKIP when missing.

Listing checks

```
preston-check --list          # list every check ID + name
preston-check --check P-712   # run one check by ID
```

```
preston-check --include-proposed   # include unreviewed community drafts
```

Versions

```
preston-check --version        # current installed version
preston-check --help           # full flag reference
```

Where to go next

- **CI examples**: `examples/`
- **Filter combinations**: `docs/all-frameworks.md`
- **Privacy + telemetry**: `docs/telemetry.md`
- **Smart contract audit module**: `modules/smart-contract-audit/README.md`
- **Issue tracker**: `https://github.com/preston-check/preston-check/issues`